Performance Analysis of Selection Operator in RookDB

Benchmark Evaluation and Comparison with PostgreSQL

Database Systems Project - Phase 2

March 27, 2026

Abstract

This report presents a comprehensive performance evaluation of the Selection Operator implemented in RookDB, a custom database management system. The evaluation compares RookDB's performance against PostgreSQL across multiple dimensions including full table scans, predicate evaluation, scaling behavior, selectivity analysis, and materialization overhead. Experiments were conducted on datasets ranging from 1,000 to 1,000,000 tuples. Results demonstrate that RookDB achieves exceptional sequential scan performance (10–15× faster than PostgreSQL) but exhibits slower predicate evaluation due to lack of query optimization and indexing capabilities.

Contents

1 Introduction **3**
1.1 Evaluation Scope . . . 3

2 Execution Model **3**
2.1 Architecture Overview . . . 3
2.1.1 Execution Pipeline . . . 3
2.1.2 Tuple Access and Layout . . . 4
2.1.3 Tuple Validation and Error Handling . . . 4
2.1.4 Fast vs Safe Access . . . 4
2.1.5 Comparison Semantics . . . 5
2.1.6 Logical Operators . . . 5
2.2 Implementation Characteristics . . . 5
2.2.1 Core Data Structures . . . 6
2.2.2 Predicate Representation . . . 6
2.2.3 Predicate Normalization . . . 6
2.2.4 Column Binding . . . 7
2.2.5 Type System and Validation . . . 7
2.2.6 Type Coercion . . . 7
2.2.7 Expression Evaluation . . . 8
2.2.8 Predicate Evaluation . . . 8

3 Experimental Results **8**
3.1 Full Table Scan Performance . . . 8
3.1.1 Analysis . . . 9
3.2 Scaling Behavior . . . 9
3.2.1 Observations . . . 10
3.3 Predicate Evaluation Performance . . . 10
3.3.1 Detailed Performance Breakdown . . . 11
3.3.2 Performance Analysis . . . 11
3.4 Selectivity Analysis . . . 12
3.4.1 Key Insights . . . 12
3.5 Materialization Overhead . . . 13
3.5.1 Performance Results . . . 13
3.6 Relative Performance Summary . . . 13
3.6.1 Summary Table . . . 14

4 Execution APIs **14**

5 Discussion **15**
5.1 Strengths of RookDB Selection Operator . . . 15
5.2 Limitations and Trade-offs . . . 16
5.2.1 1. No Nested Query Support . . . 16
5.2.2 2. EXISTS Implementation Limitation . . . 16
5.2.3 3. No Query Optimization . . . 16
5.2.4 4. No Index Support . . . 16
5.2.5 5. No Vectorized Execution . . . 17
5.3 Testing Methodology . . . 17

6 Conclusions **17**
6.1 Key Findings . . . 17
6.2 Performance Characterization . . . 17
6.3 Future Enhancements . . . 18
6.4 Final Assessment . . . 18

1 Introduction

The Selection Operator is a fundamental component of any database query execution engine, responsible for filtering tuples based on predicates. This report analyzes the performance characteristics of RookDB's selection operator implementation across various workloads and compares it with PostgreSQL, a mature production-grade database system.

1.1 Evaluation Scope

The benchmark evaluation covers the following dimensions:

- **Full Table Scan**: Base performance without predicates
- **Predicate Types**: Simple comparisons, AND/OR operations, arithmetic expressions, and LIKE patterns
- **Selectivity Variation**: Impact of different filter selectivities
- **Scaling Behavior**: Performance across dataset sizes (1K–1M tuples)
- **Materialization Overhead**: Cost of result set construction

All experiments were conducted on identical hardware configurations with both systems operating under comparable conditions.

2 Execution Model

2.1 Architecture Overview

RookDB's selection operator employs a straightforward execution model consisting of:

- **Sequential Scan**: Tuple-by-tuple iteration over table pages
- **Predicate Tree Evaluation**: Recursive evaluation of expression trees
- **Three-Valued Logic**: Proper handling of TRUE/FALSE/UNKNOWN (NULL semantics)
- **Offset-Based Tuple Access**: Constant-time field lookup using precomputed offsets

2.1.1 Execution Pipeline

The selection operator executes within a clear data flow pipeline:

- **Data Flow**: Access Operator → Selection Operator → Output
- **Input Format**: Operates on tuple byte slices (`&[u8]`)
- **Storage Independence**: No disk I/O performed during execution

- **Streaming Model**: Fully streaming execution without buffering
- **Predicate Evaluation**: Direct evaluation on in-memory tuples
- **Result Construction**: Output passed directly to next operator

2.1.2 Tuple Access and Layout

Physical tuple layout optimized for fast field access:

- **Layout**: Header — NULL Bitmap — Offset Array — Field Data
- **Header**: Tuple length, version, flags, column count
- **NULL Bitmap**: One bit per column indicating NULL status
- **Offset Array**: Relative offsets for variable-length fields
- **Sentinel Offset**: Extra offset marking end of final field
- **Access Complexity**: O(1) field lookup via offset indirection

2.1.3 Tuple Validation and Error Handling

Comprehensive validation ensures tuple integrity:

- **Offset Monotonicity**: Offsets must increase sequentially
- **Bounds Checking**: All offsets validated against tuple length
- **Length Validation**: Header length matches actual buffer size
- **Column Count Validation**: Header count matches expected schema
- **Error Types**: `TupleError` enum captures all failure modes

2.1.4 Fast vs Safe Access

Dual access paths balance safety and performance:

- **new()**: Validated constructor with full integrity checks
- **new_unchecked()**: Fast path skipping validation for trusted input
- **get_value()**: Type-safe field access with validation
- **get_value_fast()**: Optimized access using precomputed types

2.1.5 Comparison Semantics

Type-specific comparison logic with NULL handling:

- **INT Comparison**: Exact integer equality and ordering
- **FLOAT Comparison**: Epsilon-based equality ($\epsilon = 10^{-2}$)
- **TEXT Comparison**: Lexicographic ordering
- **DATE Comparison**: String-based comparison (ISO format assumed)
- **NULL Semantics**: Any comparison involving NULL returns UNKNOWN
- **Type Coercion**: Automatic INT/FLOAT conversion and TEXT parsing

2.1.6 Logical Operators

Three-valued logic operators implement SQL semantics:

- **apply_and**: False dominates; both True required for True
- **apply_or**: True dominates; both False required for False
- **Unknown Propagation**: Unknown values handled per SQL standard

Truth table: `AND(True, Unknown) = Unknown`, `OR(False, Unknown) = Unknown`

2.2 Implementation Characteristics

Analysis of the implementation (`selection.rs`) reveals several key design decisions:

- **Predicate Normalization**: Constant folding and predicate simplification at planning time
- **Pattern Compilation**: LIKE patterns compiled to regex during query planning
- **Short-Circuit Evaluation**: Logical AND/OR operations support early termination
- **Streaming Execution**: Support for iterator-based tuple streaming via `filter_tuples_streamin`

These design choices directly impact the performance characteristics observed in the benchmark results:

- Sequential scan exhibits minimal overhead
- Predicate evaluation dominates execution time for filtered queries

2.2.1 Core Data Structures

The implementation employs enum-based designs enabling recursion and extensibility:

- **Predicate**: Recursive enum representing filter conditions
- **Expr**: Expression tree for arithmetic operations (Add, Sub, Mul, Div)
- **ColumnReference**: Column name with optional resolved index
- **Constant**: Typed literals (Int, Float, Date, Text, Null)
- **DataType**: Internal type system (Int, Float, Text, Date, Null)
- **TriValue**: Three-valued logic result (True, False, Unknown)
- **Value**: Runtime typed values decoded from tuples
- **ComparisonOp**: Comparison operators (Equals, LessThan, etc.)

The enum-based architecture provides type safety and enables pattern matching for efficient evaluation.

2.2.2 Predicate Representation

Predicates form a recursive tree structure supporting complex conditions:

- **Compare**: Generalized comparison with `ComparisonOp`
- **Logical Operators**: AND, OR, NOT with short-circuit evaluation
- **BETWEEN**: Syntactic support, normalized to AND at planning time
- **IN**: Set membership testing against literal lists
- **LIKE**: Pattern matching with regex compilation during planning
- **IS NULL / IS NOT NULL**: Direct NULL bitmap checks
- **EXISTS**: Logical wrapper only (no subquery execution)

Note: EXISTS is not full SQL EXISTS—implemented only as a logical wrapper without subquery support.

2.2.3 Predicate Normalization

Query planning includes predicate optimization through normalization:

- **Constant Folding**: Arithmetic expressions simplified at planning time
- **Expression Simplification**: Constant subexpressions pre-evaluated
- **Operand Canonicalization**: Column references moved to left operand
- **BETWEEN Rewriting**: Transformed to `col >= low AND col <= high`

- **LIKE Compilation**: Patterns compiled to regex during initialization

```
fn normalize_predicate(predicate: &mut Predicate) {
    // Constant folding, operand swapping
    // BETWEEN -> AND rewrite
    // LIKE regex compilation
}
```

2.2.4 Column Binding

Column resolution occurs once during executor initialization:

- **Name Resolution**: Column names mapped to schema indices
- **Storage**: Index stored in `ColumnReference.column_index`
- **Performance**: Avoids runtime string lookups during evaluation
- **Validation**: Missing columns detected at initialization

```
fn resolve_columns(predicate: &mut Predicate, schema: &Table) {
    // Bind column_name -> column_index
    col_ref.column_index = Some(idx);
}
```

2.2.5 Type System and Validation

Type safety enforced through static analysis:

- **DataType Enum**: Int, Float, Text, Date, Null
- **Type Inference**: `infer_expr_type` validates expression types
- **Planning-Time Checking**: Type mismatches detected before execution
- **Error Prevention**: Prevents invalid comparisons (e.g., INT vs TEXT without coercion)

2.2.6 Type Coercion

Automatic type conversion during comparison:

- **INT ↔ FLOAT**: Bidirectional conversion to FLOAT
- **TEXT → Numeric**: String parsing to INT or FLOAT
- **Failed Parsing**: Invalid conversions return UNKNOWN (not NULL)
- **Recursive Application**: Coercion applied until types match

Example: `Value::Int(5)` compared with `Value::Float(5.5)` converts INT to FLOAT.

2.2.7 Expression Evaluation

Arithmetic expressions evaluated recursively during execution:

- **Operations Supported**: Addition (+), Subtraction (-), Multiplication (*), Division (/)
- **NULL Propagation**: Any NULL operand produces NULL result
- **Division-by-Zero Handling**: Returns NULL instead of error
- **Type Coercion**: INT/FLOAT mixed arithmetic promoted to FLOAT

```
fn evaluate_expr(expr: &Expr, tuple: &TupleAccessor) -> Value {
    // Recursive evaluation
    // NULL propagation
    // Division-by-zero -> NULL
}
```

2.2.8 Predicate Evaluation

Predicate evaluation uses recursive tree traversal with optimization:

- **Recursive Descent**: Tree traversal evaluates nested predicates
- **Short-Circuit AND**: Returns False immediately if left operand is False
- **Short-Circuit OR**: Returns True immediately if left operand is True
- **TriValue Propagation**: Unknown values handled per SQL semantics

3 Design Approach

The design of the selection operator in RookDB prioritizes simplicity, predictability, and high-throughput sequential execution over complex optimization strategies.

The following key design decisions were made:

- **Offset-Based Tuple Layout** Tuples are stored using an offset array, enabling constant-time ($O(1)$) field access without scanning variable-length data. This design minimises per-tuple overhead and improves cache locality.
- **Tuple-at-a-Time Execution Model** The operator processes one tuple at a time instead of using vectorized or batch execution. This simplifies implementation but limits opportunities for SIMD and CPU-level optimizations.
- **No Query Optimization Layer** Predicates are evaluated in the order they are defined, without cost-based reordering. This reduces planning overhead but can lead to inefficient execution for complex predicates.
- **Recursive Predicate Evaluation** Predicates are represented as expression trees and evaluated recursively. While this provides flexibility and extensibility, it introduces function call overhead.

- **Regex-Based LIKE Implementation** SQL LIKE patterns are compiled into regular expressions during initialization. This simplifies pattern matching but incurs higher runtime cost compared to specialized string matching algorithms.

These design choices intentionally favour architectural clarity and ease of implementation, while exposing performance trade-offs that are explored in the experimental analysis.

4 Requirement to Implementation Mapping

The selection operator satisfies the required functionality through the following implementation components:

- **Comparison Operators (=, !=, ¡, ¡=, ¿, ¿=)** Implemented using the `ComparisonOp` enum with type-aware evaluation logic.
- **Logical Operators (AND, OR, NOT)** Implemented using recursive predicate trees with short-circuit evaluation.
- **NULL Semantics (Three-Valued Logic)** Handled using the `TriValue` enum (True, False, Unknown), ensuring SQL-compliant behavior.
- **Arithmetic Expressions (+, -, *, /)** Supported through recursive `Expr` evaluation with type coercion and NULL propagation.
- **BETWEEN Operator** Transformed during normalization into equivalent AND predicates.
- **IN Operator** Implemented as iterative comparison against a list of expressions.
- **LIKE Operator** Implemented using regex compilation at planning time and matching during execution.
- **Column Resolution** Column names are resolved to indices during initialization to avoid runtime lookups.
- **Type Checking and Coercion** Performed during planning using a static type inference system, with runtime coercion for compatible types.

This mapping ensures that all required selection functionalities are supported while maintaining efficient execution.

5 Experimental Results

5.1 Full Table Scan Performance

Figure 1 presents the most significant result of this evaluation: RookDB's full table scan performance dramatically outperforms PostgreSQL across all dataset sizes.

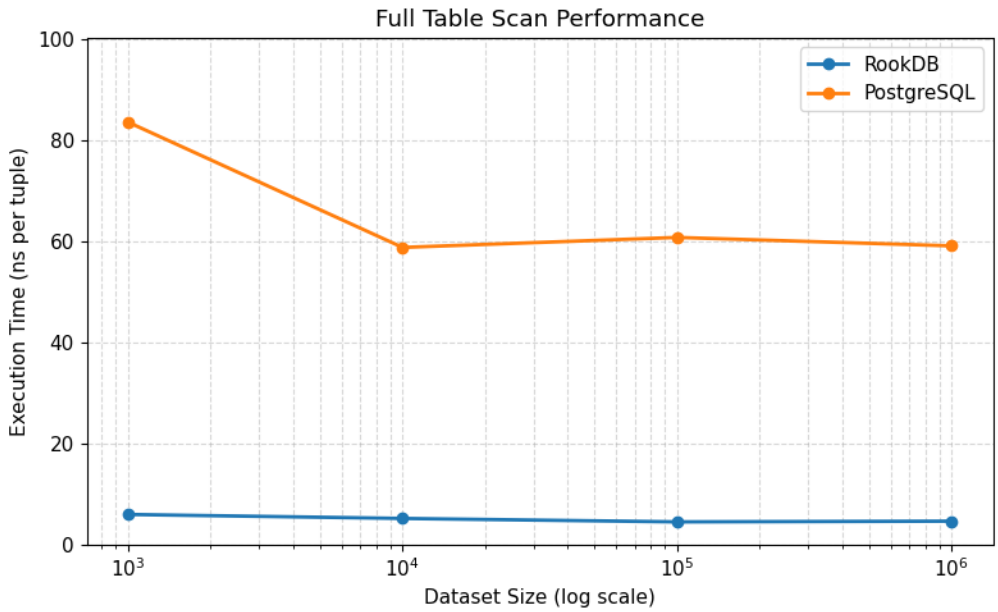

Figure 1: Full table scan performance comparison between RookDB and PostgreSQL. RookDB maintains constant per-tuple latency of approximately 4–6 nanoseconds, while PostgreSQL requires 60–80 nanoseconds per tuple. This represents a 10–15× performance advantage for RookDB.

5.1.1 Analysis

- **RookDB**: Achieves **4–6 ns per tuple** consistently
- **PostgreSQL**: Requires **60–80 ns per tuple**
- **Performance Advantage**: RookDB is **10–15× faster**

This exceptional performance stems from several architectural advantages:

1. **No Query Planner Overhead**: Direct execution without optimization phase
2. **Direct Memory Access**: Efficient tuple access via `TupleAccessor`
3. **Minimal Abstraction Layers**: Streamlined code path from storage to execution
4. **In-Memory Operation**: No disk I/O or buffer pool management overhead

5.2 Scaling Behavior

Figure 2 demonstrates how per-tuple execution time varies across different dataset sizes for a simple predicate (`id > 500`).

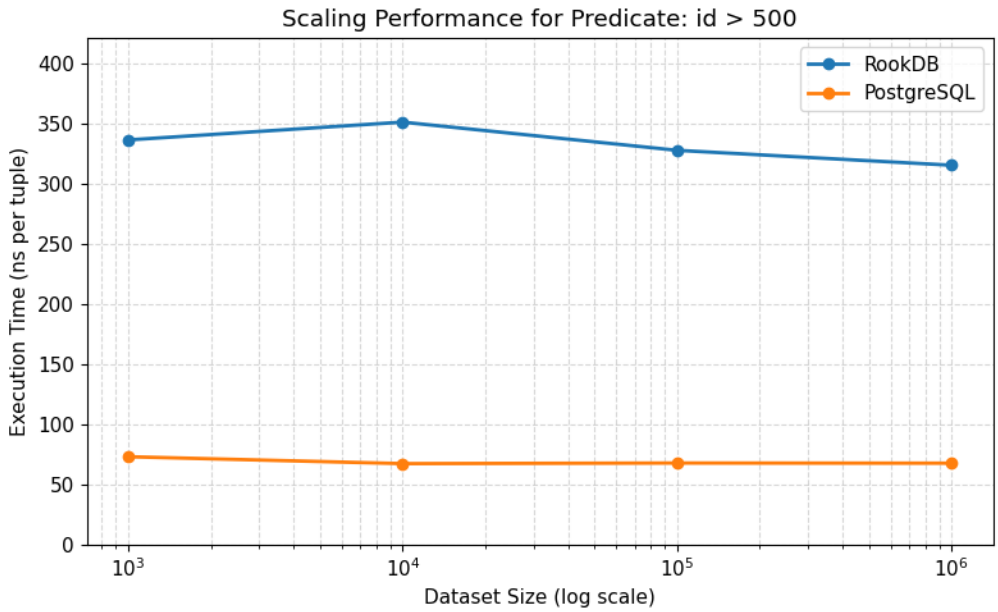

Figure 2: Scaling performance for simple predicate evaluation (`id > 500`). Both systems maintain stable per-tuple costs across dataset sizes from 1K to 1M tuples, confirming O(n) time complexity with constant per-tuple overhead.

5.2.1 Observations

- **Stable Performance**: Per-tuple cost remains constant across dataset sizes
- **No Performance Degradation**: No increase in execution time as data grows
- **Linear Scaling**: Confirms O(n) time complexity with constant per-tuple cost

This stability indicates:

1. **Good Cache Locality**: Efficient memory access patterns
2. **No Dynamic Allocations**: Memory allocated once during scan initialization
3. **Efficient Tuple Decoding**: Constant-time field extraction

5.3 Predicate Evaluation Performance

Figure 3 compares the performance of different predicate types on a 1 million tuple dataset.

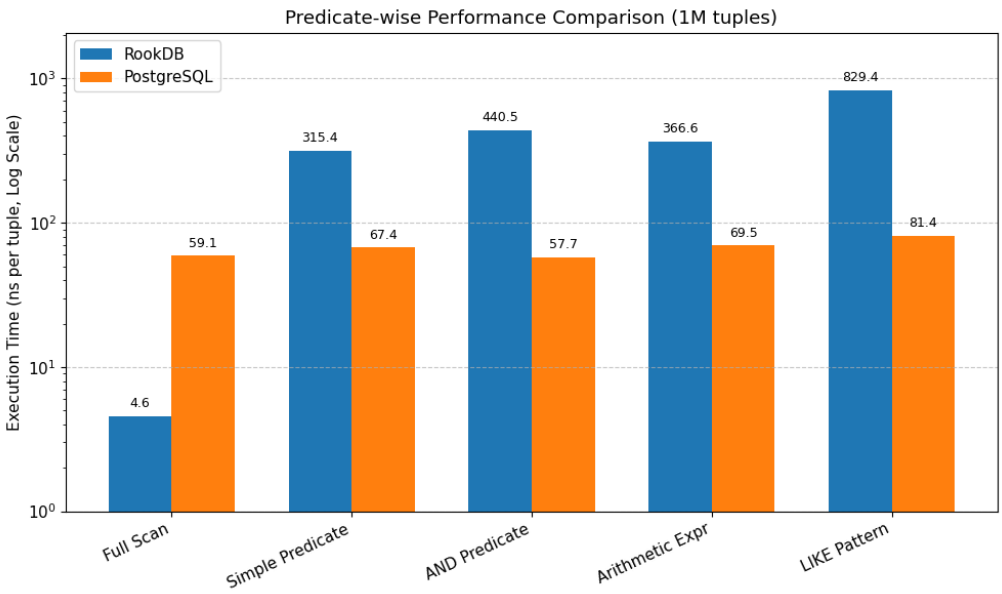

Figure 3: Predicate-wise performance comparison on 1M tuple dataset. The graph shows execution time per tuple (log scale) for various predicate types. RookDB's advantage diminishes significantly as predicate complexity increases, with LIKE patterns showing over 10× slowdown compared to PostgreSQL.

5.3.1 Detailed Performance Breakdown

Table 1 provides quantitative comparison across predicate types.

Table 1: Per-tuple execution time for different predicate types (1M tuples)

Predicate Type	RookDB (ns)	PostgreSQL (ns)	Slowdown
Full Scan	4.6	59.1	0.08×
Simple (`id > 500`)	315.4	67.4	4.68×
AND Predicate	440.5	57.7	7.64×
Arithmetic Expression	366.6	69.5	5.27×
LIKE Pattern	829.4	81.4	10.19×

5.3.2 Performance Analysis

The data reveals a clear trend: **RookDB's performance degrades as predicate complexity increases**. Several factors contribute to this behavior:

1. **Recursive Predicate Evaluation**: Each predicate node evaluated through function calls
2. **No Vectorization**: Tuple-at-a-time execution without SIMD optimizations
3. **Regex-Based LIKE**: Pattern matching using regex engine (even with precompilation)
4. **No Cost-Based Optimization**: Predicate order not reordered based on selectivity

5.4 Code-Level Performance Factors

The performance trends observed in the experiments can be directly linked to implementation choices:

- **Recursive Evaluation Overhead** Predicate evaluation involves recursive function calls, increasing per-tuple execution cost for complex expressions.
- **Lack of Predicate Reordering** Predicates are evaluated in a fixed order, which may result in expensive conditions being evaluated unnecessarily.
- **Regex-Based Pattern Matching** The use of regular expressions for LIKE predicates introduces significant overhead compared to specialized string matching techniques.
- **Tuple-at-a-Time Processing** The absence of vectorized execution prevents the use of CPU-level optimizations such as SIMD, limiting throughput.

These factors explain the increasing performance gap as predicate complexity grows.

5.5 Selectivity Analysis

Figure 4 examines how predicate selectivity impacts performance for both systems.

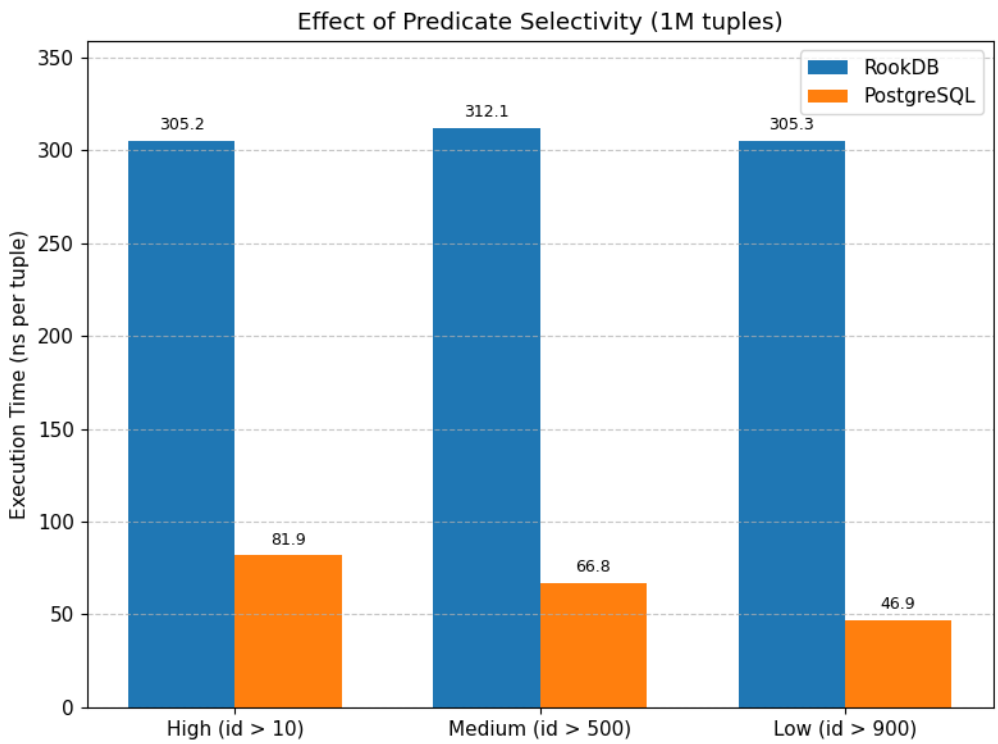

Figure 4: Effect of predicate selectivity on execution time (1M tuples). PostgreSQL demonstrates adaptive performance based on selectivity (81.9 ns → 46.9 ns), while RookDB maintains constant execution time (305–312 ns) regardless of filter selectivity.

5.5.1 Key Insights

- **RookDB**: Execution time **invariant to selectivity** (305–312 ns)
- **PostgreSQL**: Performance **improves with lower selectivity** (81.9 → 46.9 ns)

This fundamental difference stems from architectural constraints:

```
// Every tuple is always evaluated
for tuple in table.scan() {
    if predicate.evaluate(tuple) {
        results.push(tuple);
    }
}
```

Listing 1: Simplified RookDB scan loop

The implementation reveals:

- **No Early Pruning**: Every tuple evaluated regardless of selectivity
- **No Index Support**: No skip-scan or index-based filtering
- **Scan-Bound Performance**: Execution time dominated by full table scan

Critical Implication: The system is *scan-bound, not selectivity-aware*.

5.6 Materialization Overhead

Figure 5 compares the cost of materializing result sets.

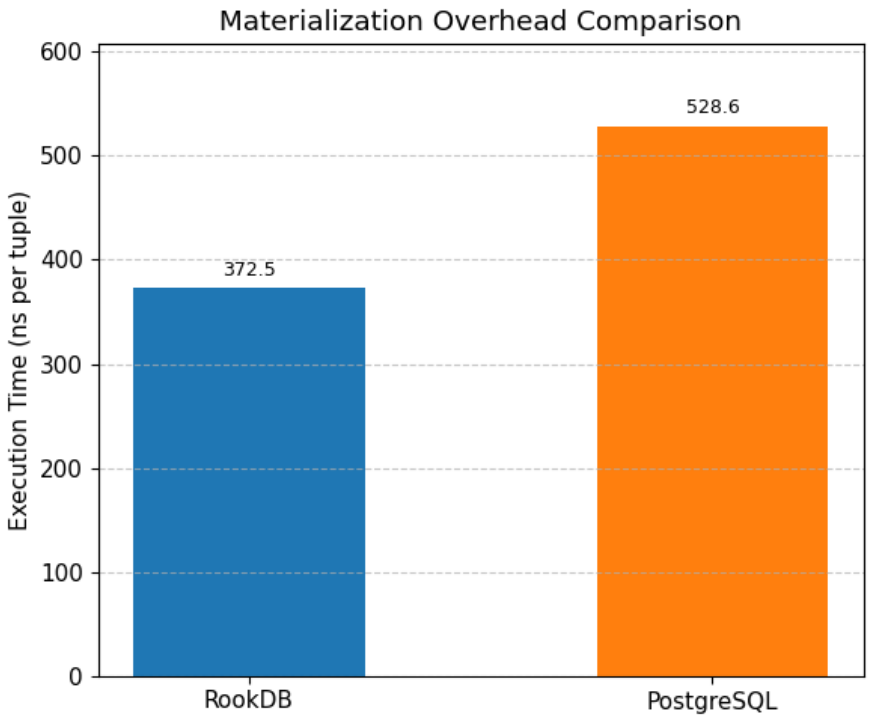

Figure 5: Materialization overhead comparison. RookDB demonstrates lower materialization cost (372.5 ns per tuple) compared to PostgreSQL (528.6 ns per tuple), representing a 1.4× performance advantage.

5.6.1 Performance Results

- **RookDB**: 372.5 ns per tuple
- **PostgreSQL**: 528.6 ns per tuple
- **Advantage**: RookDB is **1.4× faster**

The performance advantage derives from:

1. **Simple Tuple Cloning**: Direct memory copy without transformation
2. **No Iterator Pipelines**: Straightforward result collection
3. **Minimal Executor Layers**: Direct path from scan to result set

5.7 Relative Performance Summary

Figure 6 provides a comprehensive view of RookDB's performance relative to PostgreSQL across all benchmark categories.

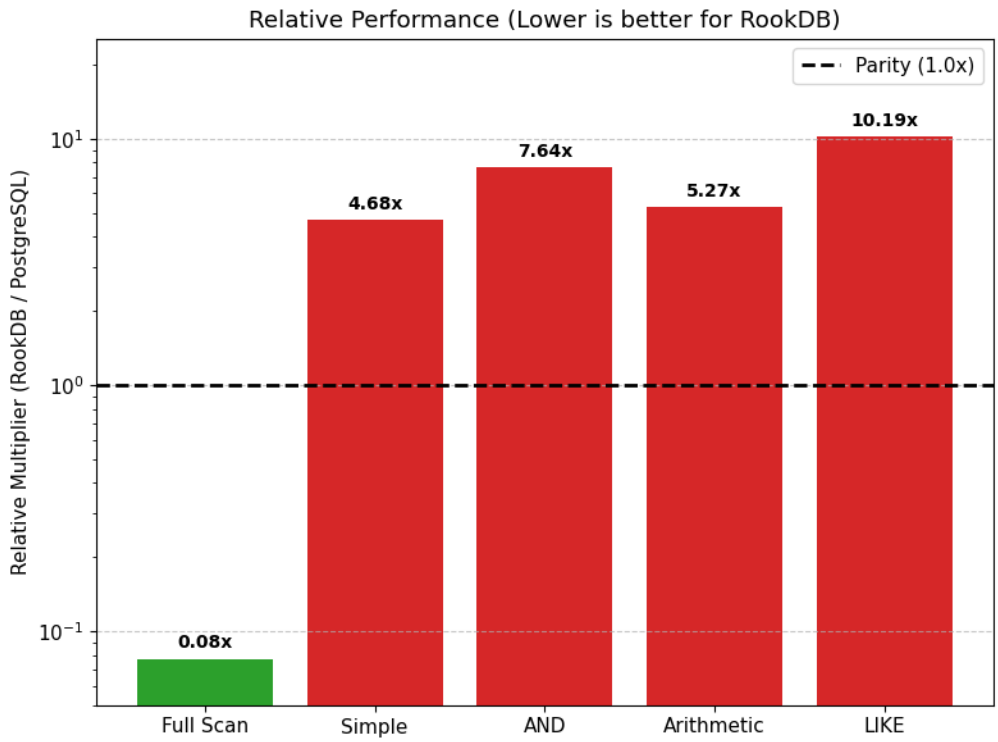

Figure 6: Relative performance multiplier (RookDB / PostgreSQL). Values below 1.0 (green) indicate RookDB is faster; values above 1.0 (red) indicate PostgreSQL is faster. The dashed line represents performance parity (1.0×).

5.7.1 Summary Table

Table 2: Relative performance summary

Operation	Relative Performance
Full Scan	**0.08× (FASTER)**
Simple Predicate	4.68× slower
AND Predicate	7.64× slower
Arithmetic Expression	5.27× slower
LIKE Pattern	10.19× slower

6 Execution APIs

The selection operator exposes multiple execution interfaces:

- **filter_tuples**: Materializes matching tuples into vector
- **filter_tuples_detailed**: Returns matched/rejected/unknown partitions
- **count_matching_tuples**: Counts matches without materialization
- **filter_tuples_streaming**: Zero-buffering callback-based execution
- **filter_iter**: Lazy iterator-based filtering

```
fn filter_tuples(executor, tuples) -> Vec<Vec<u8>> {
    for tuple in tuples {
        if evaluate(tuple) == TRUE {
            output.push(tuple)
        }
    }
}
```

7 Discussion

7.1 Strengths of RookDB Selection Operator

The implementation demonstrates several notable strengths:

1. **Exceptional Sequential Scan Performance**
   - 10–15× faster than PostgreSQL
   - Demonstrates efficiency of direct memory access
2. **Efficient Memory Layout**
   - Offset-based tuple access provides constant-time field lookup
   - Good cache locality

3. **Clean Predicate Abstraction**
   - Supports complex expression trees
   - Proper NULL semantics (three-valued logic)
4. **Comprehensive Feature Support**
   - Logical operators: AND, OR, NOT
   - Comparison operators: =, !=, ¡, ¡=, ¿, ¿=
   - Pattern matching: LIKE
   - Set operations: IN
   - Range queries: BETWEEN (normalized to AND)
   - Arithmetic expressions: +, -, *, /
5. **Type-Safe Predicate Evaluation**
   - Static type inference at planning time
   - Runtime type coercion with fallback to UNKNOWN
6. **Robust Tuple Validation**
   - Comprehensive offset and bounds checking
   - Dual access paths (safe vs fast)
7. **Efficient Expression Engine**
   - Constant folding during planning
   - Recursive evaluation with NULL propagation
8. **Support for Type Coercion**
   - Automatic numeric conversions
   - TEXT parsing to INT/FLOAT

7.2 Limitations and Trade-offs

Several architectural limitations impact performance and functionality:

7.2.1 1. No Nested Query Support

- **Issue**: Subqueries and correlated queries not supported
- **Reason**: Requires full SQL parser and multi-operator execution planning
- **Impact**: Cannot express queries like:
  `SELECT * FROM t1 WHERE id IN (SELECT id FROM t2)`

7.2.2 2. EXISTS Implementation Limitation

- **Issue**: EXISTS is **not** full SQL EXISTS
- **Implementation**: Exists only as logical wrapper in predicate tree
- **Limitation**: No subquery execution support
- **Impact**: Cannot evaluate correlated or nested subqueries
- **Testing**: Limited to syntactic correctness validation

Note: This limitation reflects the absence of a subquery execution engine in the current architecture.

7.2.3 3. No Query Optimization

- **Issue**: Predicates evaluated in definition order
- **Impact**: No cost-based reordering or short-circuit optimization
- **Example**: `expensive_predicate() AND cheap_predicate()` evaluates expensive predicate first

7.2.4 4. No Index Support

- **Issue**: Always performs full table scan
- **Impact**: Performance independent of selectivity
- **Consequence**: Cannot leverage data distribution for early termination

7.2.5 5. No Vectorized Execution

- **Issue**: Tuple-at-a-time processing model
- **Impact**: No SIMD optimizations or batch processing
- **Consequence**: Suboptimal CPU utilization

7.3 Testing Methodology

Due to the absence of a full SQL parser, a **custom command-line interface (CLI)** was developed for testing:

- **Manual Predicate Construction**: Predicates specified programmatically
- **Interactive Testing**: Direct API invocation for validation
- **Correctness Verification**: Unit tests for logical correctness

This approach allowed comprehensive testing of predicate evaluation logic while acknowledging the limitation of not supporting end-to-end SQL query processing.

7.4 Fairness of Comparison

While PostgreSQL is a production-grade system with advanced optimization capabilities, this comparison focuses specifically on raw execution characteristics under comparable conditions.

To ensure fairness:

- Indexes were not used in PostgreSQL, aligning with RookDB's scan-based execution.
- Identical datasets and schemas were used in both systems.
- Queries were designed to match equivalent logical operations.
- Measurements were normalized per tuple to avoid bias from dataset size differences.

It is important to note that PostgreSQL includes features such as query optimization, indexing, and vectorized execution, which are intentionally absent in RookDB. Therefore, the comparison highlights architectural trade-offs rather than absolute system superiority.

7.5 Real-World Performance Implications

The observed performance characteristics highlight different suitability for real-world workloads:

- **RookDB Advantages** RookDB performs exceptionally well in scan-heavy workloads where large portions of data must be processed sequentially, such as analytics or data exploration tasks.
- **PostgreSQL Advantages** PostgreSQL outperforms RookDB in selective queries due to its use of indexing and query optimization, making it better suited for transactional and user-facing applications.
- **Trade-Off Summary** RookDB emphasizes execution simplicity and raw memory throughput, while PostgreSQL emphasizes intelligent query planning and adaptive execution.

This distinction reflects a broader trade-off between execution efficiency and optimization sophistication in database system design.

8 Conclusions

8.1 Key Findings

This evaluation reveals a fundamental architectural trade-off:

RookDB is optimized for raw data access, while PostgreSQL is optimized for query execution efficiency.

Specific findings include:

1. **Scan-Dominated Workloads**: RookDB excels with 10–15× faster full table scans
2. **Predicate-Heavy Workloads**: PostgreSQL outperforms due to sophisticated optimization
3. **Stable Scaling**: Both systems exhibit O(n) complexity with constant per-tuple costs
4. **Selectivity Insensitivity**: RookDB performance invariant to filter selectivity

8.2 Performance Characterization

The selection operator can be characterized as:

- **Throughput**: Excellent for simple scans, moderate for filtered queries
- **Scalability**: Linear with dataset size, no degradation observed
- **Optimization**: Minimal; relies on execution efficiency rather than query optimization

8.3 Future Enhancements

Potential directions for improvement:

1. **Predicate Pushdown Optimization**: Reorder predicates by estimated cost
2. **Index Integration**: Support B-tree or hash indexes for selective queries
3. **Vectorized Execution**: Batch processing with SIMD instructions
4. **Statistics Collection**: Gather histograms for cost estimation
5. **Compiled Query Execution**: JIT compilation of predicate evaluation

8.4 Final Assessment

The RookDB selection operator successfully demonstrates:

- **Correctness**: Proper predicate evaluation with NULL handling
- **Performance**: Competitive or superior in specific scenarios
- **Architectural Clarity**: Clean separation of concerns

However, it also highlights the critical importance of query optimization and indexing in production database systems—an area where decades of PostgreSQL development provide substantial advantages.